UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería

Departamento de Ingeniería Electrónica, Mecatrónica y Biomédica



IE3028 - Simulación y Fabricación de Circuitos Impresos

Proyecto Final

Robot Sumo KRAKEN

Documentación final del diseño, prototipo, fabricación y funcionamiento

Autor

Jose Martínez - Carné 24050

Profesor

Pablo D. Mazariegos

Guatemala, octubre de 2026

Índice

1. Introducción y objetivos **2**

1.1. Objetivo general 2

1.2. Objetivos específicos 2

2. Evolución del proyecto y cambios realizados **3**

3. Método y diseño del sistema **3**

3.1. Arquitectura general 3

3.2. Módulo de alimentación 4

3.3. Módulo de control BLE 5

3.4. Driver y motores 6

3.5. Simulación y validación previa 7

4. Resultados experimentales del prototipo **7**

4.1. Construcción del prototipo 7

4.2. Pruebas funcionales del prototipo 8

4.3. Diseño mecánico preliminar 9

5. Diseño de PCB y archivos asociados **9**

5.1. Layout final 9

5.2. Reglas principales de diseño 11

5.3. Verificación DRC 11

5.4. Archivos de fabricación 12

6. BOM final **13**

7. Proceso de fabricación y ensamblaje **13**

7.1. Fabricación de la PCB 13

7.2. Soldadura 13

7.3. Ensamblaje de módulos 14

7.4. Pruebas eléctricas 15

7.5. Problemas encontrados y soluciones 16

7.6. Fabricación e integración del chasis 16

8. Firmware y software **17**

8.1. Herramientas y dependencias 17
8.2. Funciones principales del firmware 17
8.3. Instrucciones de compilación y ejecución 17

9. Resultado final y registro de pruebas **18**
9.1. Registro consolidado de pruebas 19
9.2. Evidencia en video 19

10. Limitaciones y mejoras futuras **19**

11. Lecciones aprendidas **20**

12. Recomendaciones para producción a escala **21**

13. Conclusiones **21**

14. Referencias **21**

A. Archivos digitales de la entrega **23**

B. Firmware completo **24**

C. Evidencia multimedia **27**

1. Introducción y objetivos

KRAKEN es un robot tipo sumo desarrollado como proyecto del curso IE3028. El objetivo del proyecto fue diseñar, probar, fabricar y ensamblar un sistema electrónico propio capaz de controlar dos motores y permitir el movimiento del robot mediante Bluetooth Low Energy (BLE) desde un teléfono celular.

El proyecto se trabajó por etapas. Primero se definió una propuesta general y se identificaron los principales retos. Después se construyó un prototipo en protoboard para comprobar que el ESP32-C3, el driver TB6612FNG, los motores y la comunicación BLE funcionaran correctamente. Con esa validación se desarrollaron los esquemáticos y la PCB final en Altium Designer. Finalmente se fabricó la placa, se soldaron y probaron los componentes, se integró el sistema en un chasis de MDF y se realizaron pruebas finales de funcionamiento.

Esta documentación reúne las diferentes etapas en un solo informe y muestra los cambios realizados durante el desarrollo. No se realizaron simulaciones eléctricas en software; la validación previa se hizo mediante un prototipo físico funcional.

1.1. Objetivo general

Diseñar, fabricar y validar el sistema electrónico de un robot sumo controlado por BLE, integrando el control, la potencia, la alimentación y los motores en una PCB propia instalada dentro del chasis final del robot.

1.2. Objetivos específicos

- Controlar dos motores N20 de 6 V mediante un driver TB6612FNG.
- Utilizar un ESP32-C3 como unidad de control y comunicación BLE.
- Validar el funcionamiento del sistema mediante un prototipo físico antes de fabricar la PCB.
- Diseñar una PCB que cumpla con las reglas necesarias para su fabricación en la LPKF ProtoMat S103.
- Implementar un sistema de alimentación con dos baterías 18650, protección BMS y regulación a 5 V.
- Fabricar, soldar y verificar la PCB antes de integrarla al robot.
- Integrar la electrónica, baterías, motores, ruedas e interruptor dentro de un chasis de MDF.
- Documentar los problemas encontrados, las correcciones realizadas y las mejoras posibles para una siguiente versión.

2. Evolución del proyecto y cambios realizados

La propuesta inicial definió a KRAKEN como un robot compacto controlado desde un celular. Se propusieron dos motores N20, un ESP32-C3, un TB6612FNG y cuatro sensores TCRT5000 para detectar el borde del Dohyo. También se planteó inicialmente una batería LiPo 2S y un diseño mecánico con cuñas para atacar al avanzar o retroceder.

Durante el desarrollo se realizaron cambios por disponibilidad de componentes, presupuesto, facilidad de fabricación y resultados obtenidos en el prototipo. La Tabla 1 resume los cambios principales.

Cuadro 1: Cambios principales desde la propuesta inicial hasta el sistema final.

Elemento	**Propuesta o prototipo inicial**	**Resultado final**
Alimentación	Batería LiPo 2S. En el prototipo se utilizó una fuente provisional de 5 V.	Dos baterías 18650, BMS 2S, switch y regulador XL4005 ajustado a 5 V.
Sensores de borde	Cuatro sensores TCRT5000 para detectar la línea blanca.	No se integraron en la versión final debido al presupuesto disponible.
Implementación electrónica	Prototipo sobre protoboard.	PCB propia de aproximadamente 105 mm × 75 mm.
Control	ESP32-C3 con BLE y control desde celular.	Se mantuvo el ESP32-C3 y el control BLE mediante Bluefruit Connect.
Motores	Dos motores N20 de 6 V.	Dos motores N20 de 6 V y 300 rpm, controlados por TB6612FNG.
Estructura	Diseño mecánico preliminar con cuñas.	Chasis final de MDF fabricado en D-Hive e integrado con PCB, baterías, motores, ruedas y switch.

Estos cambios no alteraron el objetivo principal del proyecto: obtener un robot funcional controlado de forma remota y con una PCB diseñada y fabricada específicamente para KRAKEN.

3. Método y diseño del sistema

3.1. Arquitectura general

El sistema final se dividió en dos bloques principales: control y movimiento, y alimentación/-carga/protección. El teléfono envía comandos BLE al ESP32-C3. El microcontrolador genera las señales digitales y PWM necesarias para el TB6612FNG, que controla los dos motores

N20. Por otro lado, las baterías y el cargador se conectan al BMS, luego al interruptor general y finalmente al XL4005, que entrega una salida regulada de 5 V.

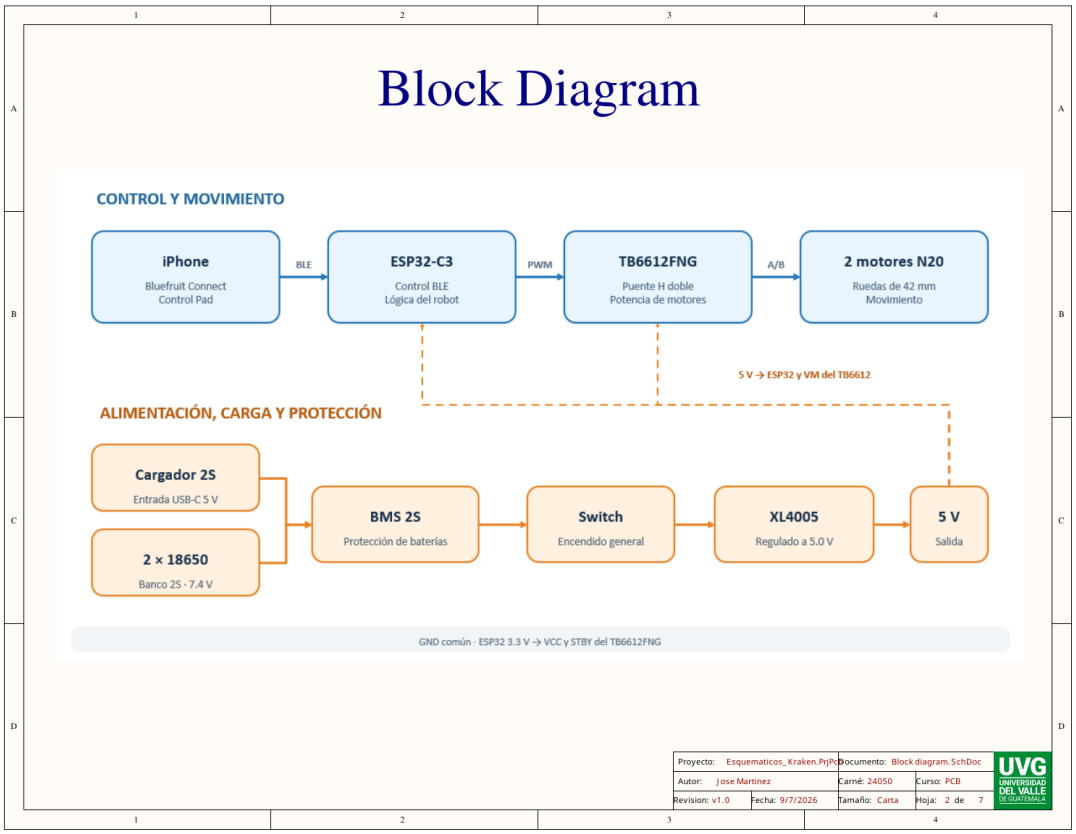


Figura 1: Diagrama de bloques final del sistema KRAKEN.

3.2. Módulo de alimentación

El sistema final utiliza dos baterías 18650 conectadas como banco 2S. El BMS 2S se encarga de la protección del banco de baterías y el módulo cargador USB-C permite realizar la carga. Después del BMS se colocó un switch general y un regulador XL4005. La salida del XL4005 se ajustó a aproximadamente 5 V antes de conectar los demás módulos.

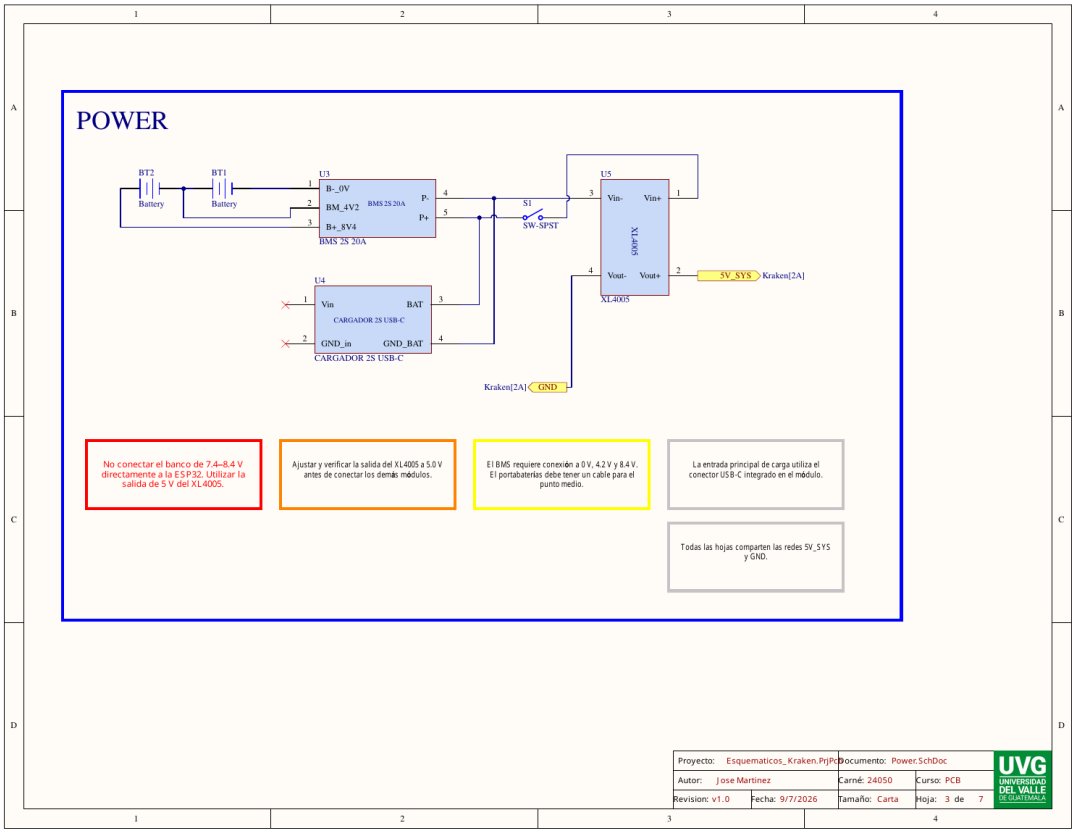


Figura 2: Esquemático final del módulo de alimentación.

3.3. Módulo de control BLE

El ESP32-C3 recibe los comandos enviados desde el teléfono. El control se realiza mediante BLE y la aplicación Bluefruit Connect. El microcontrolador trabaja con lógica de 3.3 V y genera las señales de dirección y PWM utilizadas por el driver de motores.

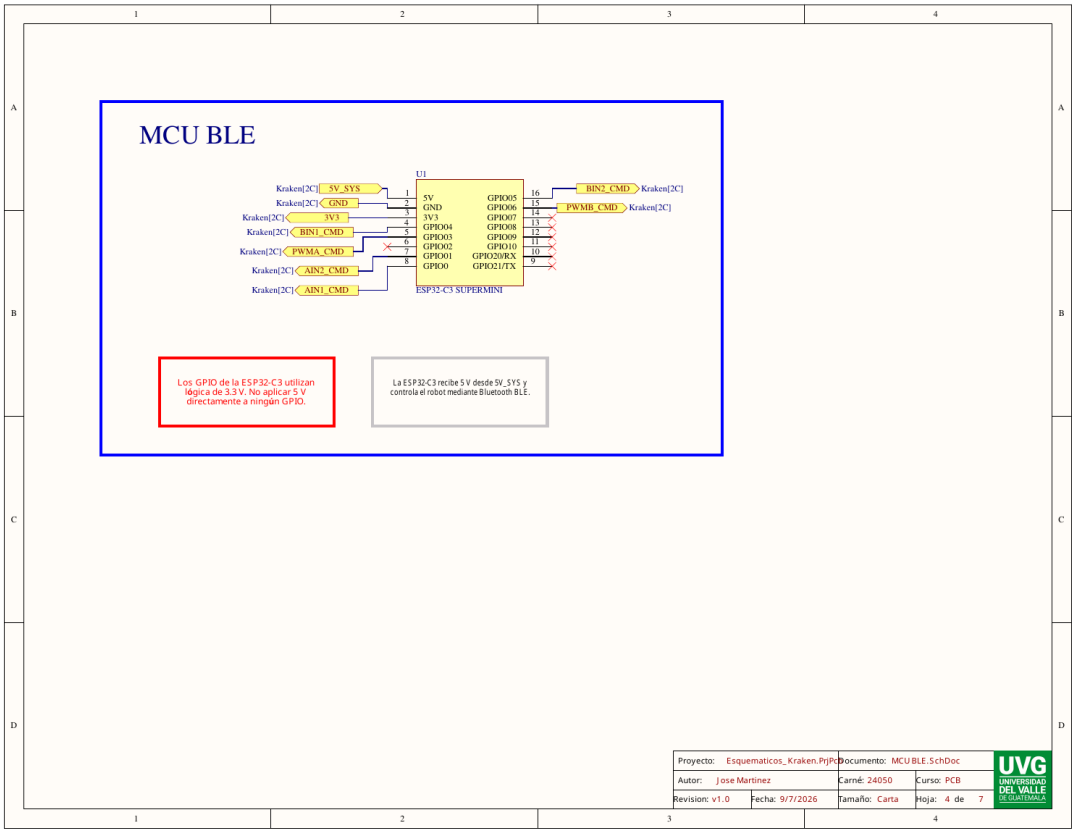


Figura 3: Esquemático del módulo ESP32-C3 y las señales de control BLE.

3.4. Driver y motores

El TB6612FNG controla dos motores N20. El driver recibe las señales de dirección y PWM del ESP32-C3. Se utiliza un capacitor de 1000 μF en la alimentación principal y dos capacitores de 100 nF asociados a los motores para ayudar a reducir ruido eléctrico y caídas de tensión observadas durante las primeras pruebas.

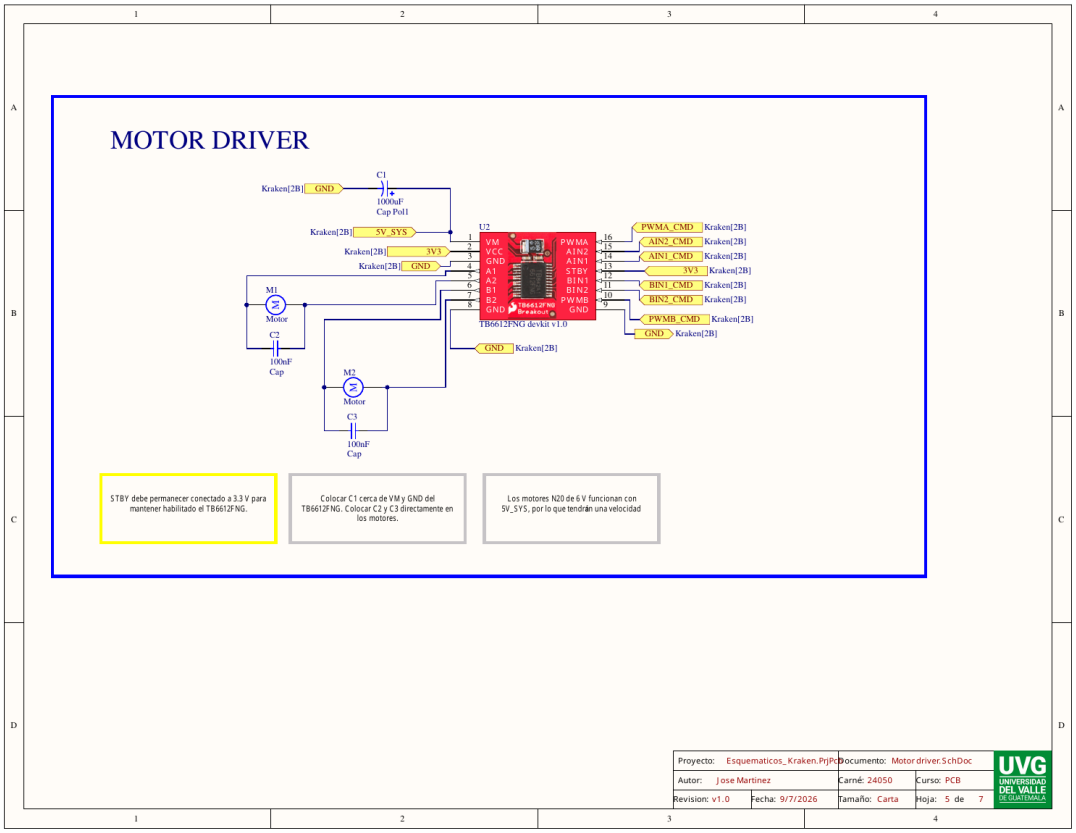


Figura 4: Esquemático final del driver TB6612FNG y los dos motores N20.

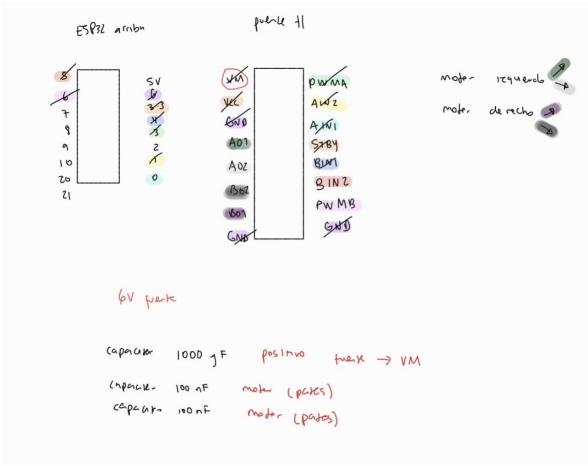
3.5. Simulación y validación previa

No se realizaron simulaciones eléctricas mediante LTspice, Proteus, Multisim u otro software similar. En este proyecto la validación previa se realizó directamente con un prototipo físico en protoboard. Esta decisión permitió probar el comportamiento real del microcontrolador, driver, motores, capacitores y comunicación BLE antes de pasar al diseño definitivo de la PCB.

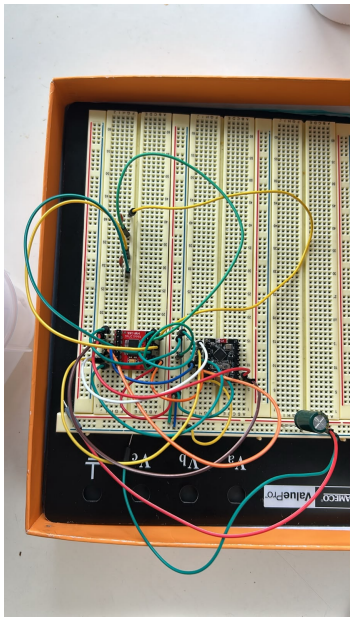
4. Resultados experimentales del prototipo

4.1. Construcción del prototipo

El primer prototipo funcional se construyó en protoboard utilizando un ESP32-C3, un TB6612FNG, dos motores N20 y los capacitores de desacoplamiento. Para esta etapa se utilizó una alimentación provisional de 5 V. También se preparó un borrador de conexiones antes del montaje físico.



(a) Borrador de conexiones utilizado durante el prototipo.



(b) Prototipo implementado en protoboard.

Figura 5: Etapa de validación física antes del diseño definitivo de la PCB.

4.2. Pruebas funcionales del prototipo

El prototipo permitió verificar las funciones básicas antes de fabricar la placa. Los resultados registrados se muestran en la Tabla 2.

Cuadro 2: Pruebas realizadas con el prototipo.

Prueba	**Resultado**
Conexión BLE	Exitosa
Avance	Correcto
Retroceso	Correcto
Giro a la izquierda	Correcto
Giro a la derecha	Correcto
Control de velocidad	Correcto
Parada por desconexión BLE	Correcta

Durante esta etapa se detectó ruido eléctrico y caídas de tensión. Como acción correctiva se añadieron dos capacitores de 100 nF y un capacitor de 1000 μF. También se implementó una parada automática de los motores cuando se pierde la comunicación BLE.

El video correspondiente a esta etapa se encuentra en:

https://youtube.com/shorts/PRTlkrRZpyA?si=jQyeP_-DCtMkYQhl

4.3. Diseño mecánico preliminar

Durante el prototipo también se realizó un modelo preliminar del chasis. Posteriormente este diseño evolucionó hacia las piezas finales fabricadas en MDF.

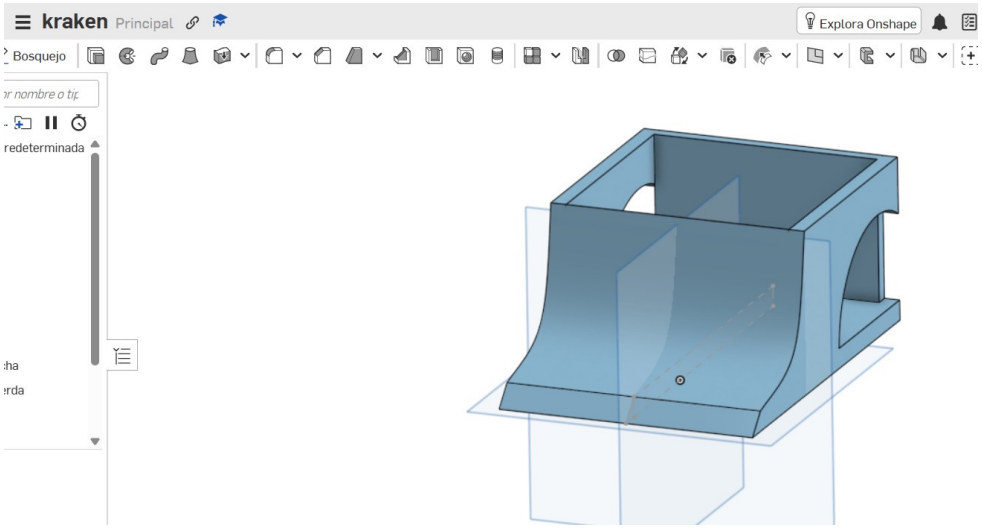


Figura 6: Modelo preliminar del chasis utilizado durante el desarrollo.

5. Diseño de PCB y archivos asociados

5.1. Layout final

La PCB final fue diseñada en Altium Designer y tiene dimensiones aproximadas de 105 mm × 75 mm. Se utilizaron cuatro agujeros de montaje de 3 mm en las esquinas. El ruteo principal se realizó en Bottom Layer para facilitar la fabricación y la soldadura.

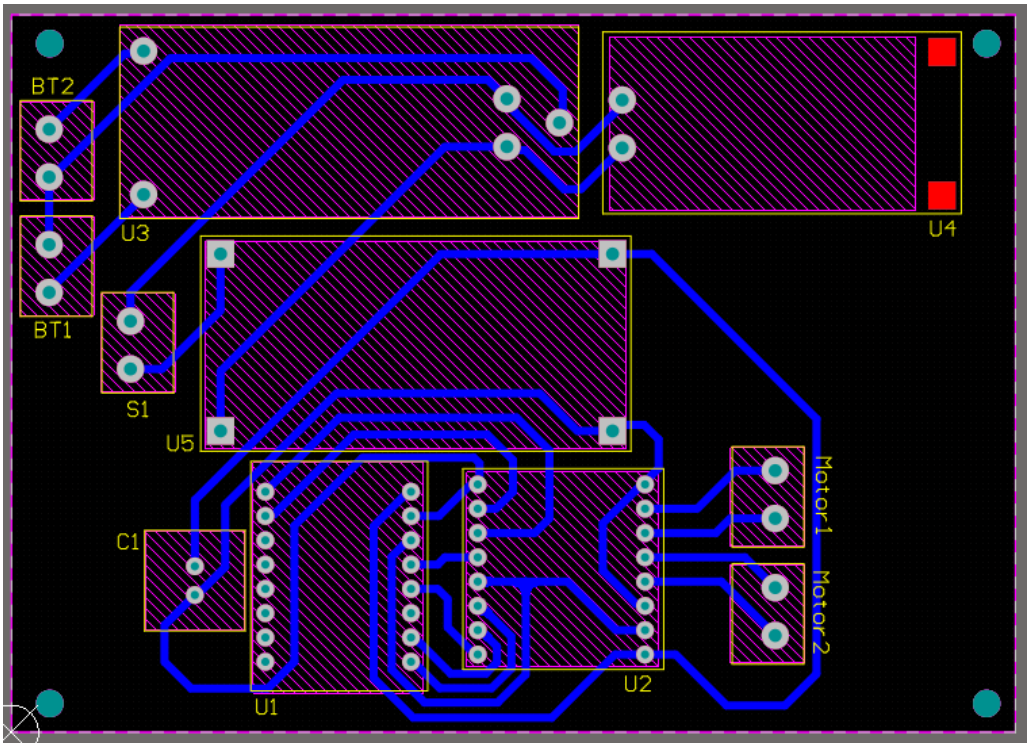


Figura 7: Layout final de la PCB de KRAKEN.

Los footprints se verificaron mediante la vista 3D para comprobar la orientación de módulos, borneras y conectores antes de generar los archivos de fabricación.

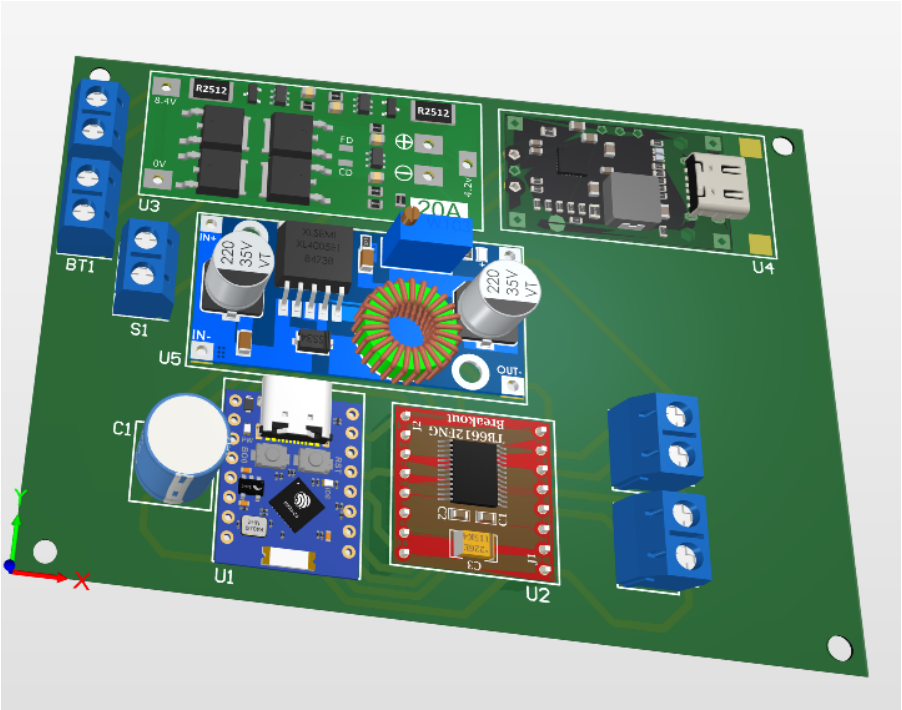


Figura 8: Vista 3D del diseño final de la PCB.

5.2. Reglas principales de diseño

Las reglas utilizadas se definieron considerando los requerimientos funcionales y de fabricación de la ProtoMat S103. Los valores principales se resumen en la Tabla 3.

Cuadro 3: Reglas principales del diseño.

Parámetro	**Valor utilizado**
Dimensiones de la placa	105 mm × 75 mm
Ancho de pista principal	1.0 mm (aprox. 39.4 mil)
Clearance	0.508 mm
Anillo anular mínimo	0.254 mm
Agujeros de componentes	1.0 mm y 1.5 mm
Agujeros de montaje	3.0 mm, NPTH
Capa principal de ruteo	Bottom Layer

Para justificar el ancho de pista se utilizaron los cálculos realizados con la herramienta ANSI PCB Trace Width basada en IPC-2221A. Para 1 A se obtuvo un ancho mínimo aproximado de 0.31 mm y para 2 A un ancho aproximado de 0.80 mm. Por esta razón se utilizó 1.0 mm en las pistas principales, proporcionando margen para ambos casos.

5.3. Verificación DRC

Al terminar el ruteo se ejecutó el Design Rule Check de Altium Designer. El resultado final fue de **0 warnings** y **0 rule violations**. Esta revisión confirmó que el diseño cumplía las reglas configuradas antes de fabricar la placa.


Designer

esign Rule Verification Report

ate: 10/31/2026
me: 11:49:50 AM
apsed Time: 00:00:00
lename: C:\Users\josem\Documents\SEMESTRE 6\PCB\Esquematicos_Kraken\Esquematicos_Kraken\PCBKraken.PcbDoc

Warnings: 0
Rule Violations: 0

ummary

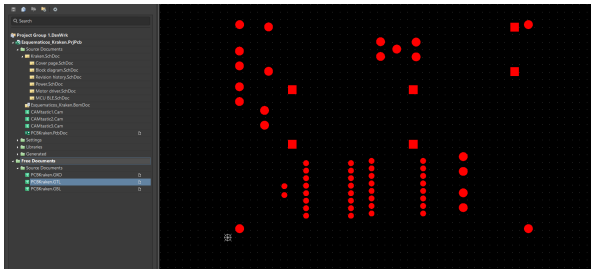
arnings	Count
Total	0

ule Violations	Count
earance Constraint (Gap=0.508mm) (All),(All)	0
ort-Circuit Constraint (Allowed=No) (All),(All)	0
n-Routed Net Constraint ((All))	0
odified Polygon (Allow modified: No), (Allow shelved: No)	0
idth Constraint (Min=0.508mm) (Max=1mm) (Preferred=0.508mm) (All)	0
idth Constraint (Min=1mm) (Max=1.5mm) (Preferred=1mm) (InNet('NetBT1_1') Or InNet('NetBT1_2') Or InNet('NetBT2_1') Or InNet('NetS1_1') Or Net('NetS1_2') Or InNet('NetU3_4') Or InNet('NetC1_1') Or InNet('NetC1_2') Or InNet('NetMotor1_1') Or InNet('NetMotor1_2') Or	0

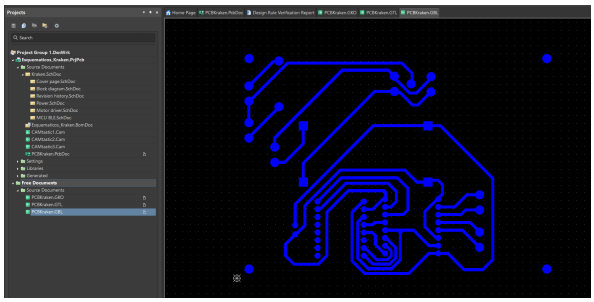
Figura 9: Reporte final de DRC con cero advertencias y cero violaciones.

5.4. Archivos de fabricación

Después de verificar el diseño se generaron los archivos Gerber y NC Drill. Los archivos principales fueron GTL, GBL y GKO. El GBL contiene las pistas del Bottom Layer y el GKO define el contorno de la placa. También se generaron archivos separados para agujeros plated y NPTH.



(a) GTL - Top Layer.



(b) GBL - Bottom Layer.

Figura 10: Archivos Gerber de cobre.

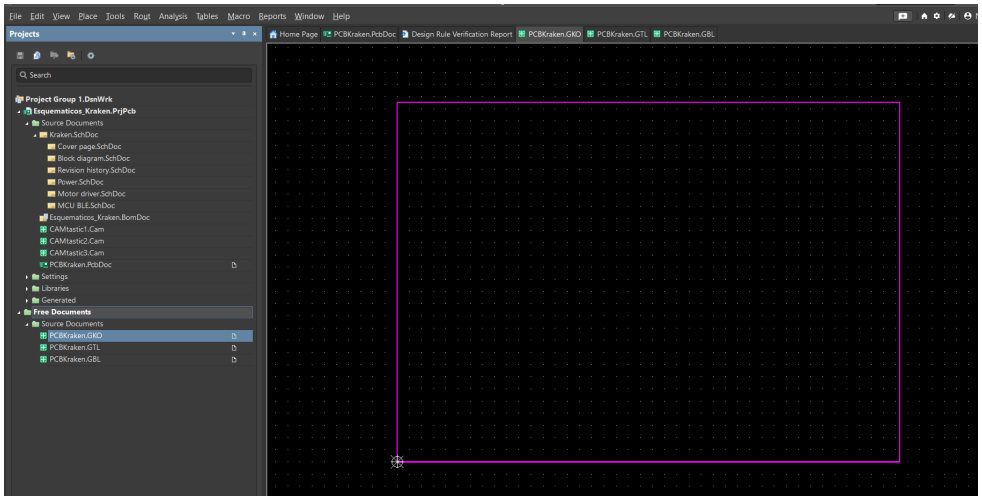
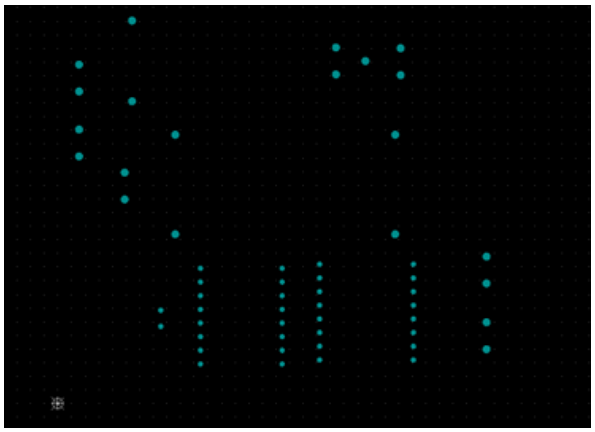
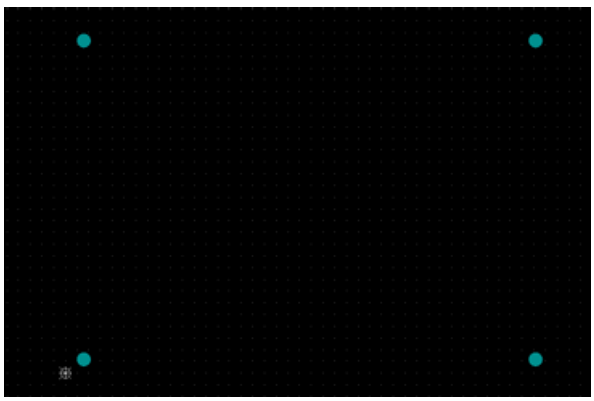


Figura 11: Archivo GKO con el contorno de la PCB.



(a) NC Drill plated.



(b) NC Drill NPTH.

Figura 12: Archivos de perforación del diseño.

6. BOM final

El BOM final corresponde al sistema electrónico utilizado en KRAKEN. Los precios fueron registrados con el proveedor DIY ELECTRONICA. El costo total de los componentes incluidos en el BOM es de **Q404.00**.

Name	Description	Designator	Value	Quantity	Supplier	SKU	Precio unitario (Q)	Subtotal (Q)
Battery	Multicell Battery	BT1, BT2		2	DIYELECTRONICA	T35	55	110
BMS2S20A	BMS2S20A v1.0	U3		1	DIYELECTRONICA	I239	29.5	29.5
Cap	Capacitor	C2, C3	100nF	2	DIYELECTRONICA	K305-1	0.5	1
Cap Pol1	Polarized Capacitor (Radial)	C1	1000uF	1	DIYELECTRONICA	K510-1	4.5	4.5
CARGADOR2S USB-C	CARGADOR2S USB-C v1.0	U4		1	DIYELECTRONICA	I232	15	15
ESP32-C3 SUPERMINI	ESP32-C3 SUPERMINI V1.0	U1		1	DIYELECTRONICA	B207	75	75
Motor	Motor N20 6V 300 RPM	M1, M2		2	DIYELECTRONICA	M225	44	88
SW-SPST	Single-Pole, Single-Throw Switch	S1		1	DIYELECTRONICA	M234	6	6
TB6612FNG devkit v1.0	TB6612FNG devkit v1.0	U2		1	DIYELECTRONICA	I235	45	45
XL4005	XL4005_MODULE	U5		1	DIYELECTRONICA	I26	30	30
Z. TOTAL DEL PROYECTO				1				Q404.00

Figura 13: BOM final exportado desde Altium Designer.

7. Proceso de fabricación y ensamblaje

7.1. Fabricación de la PCB

Los archivos Gerber y NC Drill se enviaron al profesor para realizar la fabricación mediante la LPKF ProtoMat S103. Antes de la fabricación, el diseño final fue revisado y autorizado por el profesor. En esta revisión se verificó la calidad de los tracks y que el trazado no presentara errores visibles o uniones no deseadas.

Una vez recibida la PCB se verificó la continuidad de las conexiones principales utilizando un multímetro. Después de confirmar el estado de la placa se inició la soldadura.

7.2. Soldadura

Primero se colocaron los elementos fijos, como borneras, headers y capacitor. Para cada unión se procuró calentar correctamente el pad y la terminal antes de aplicar el estaño. Se utilizó la cantidad necesaria para obtener uniones firmes y evitar puentes entre pads o pistas cercanas. Al finalizar se realizó una inspección visual.

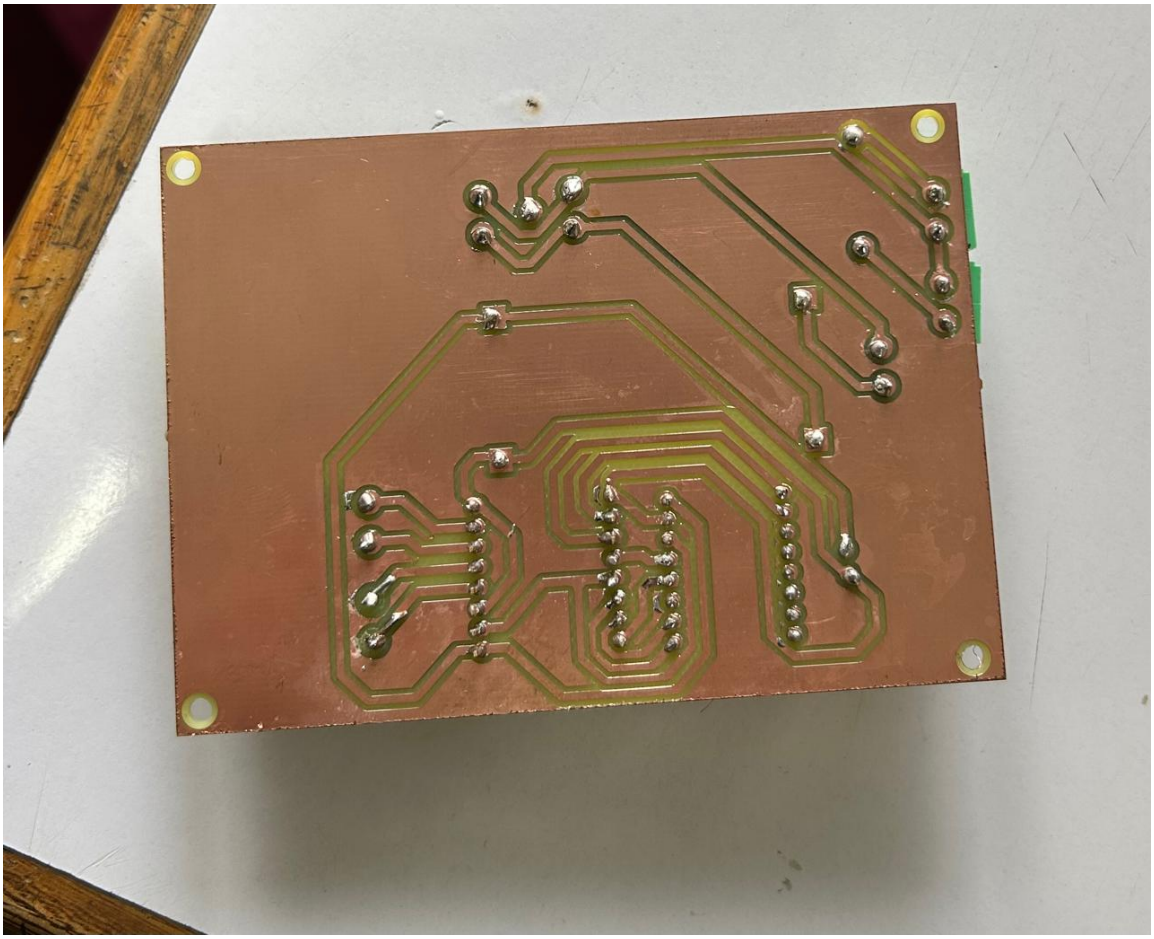
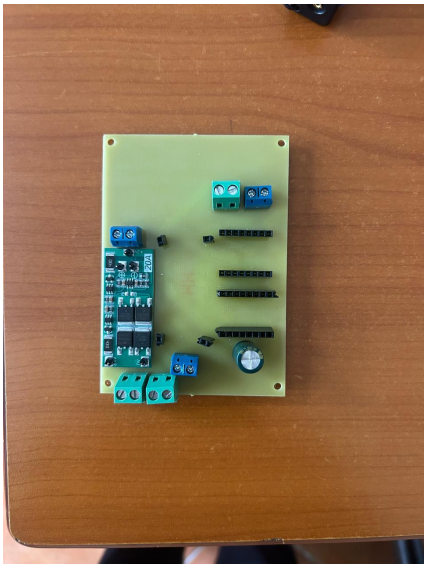


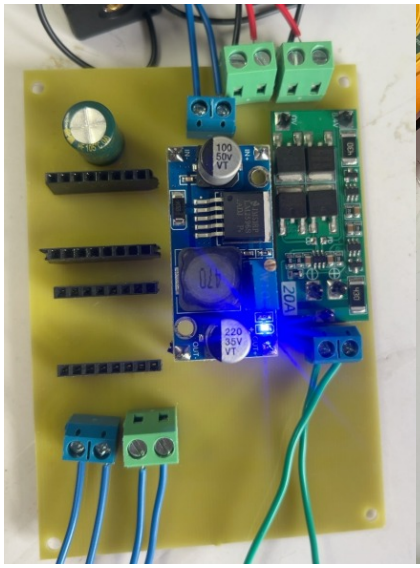
Figura 14: Vista inferior de la PCB después del proceso de soldadura.

7.3. Ensamblaje de módulos

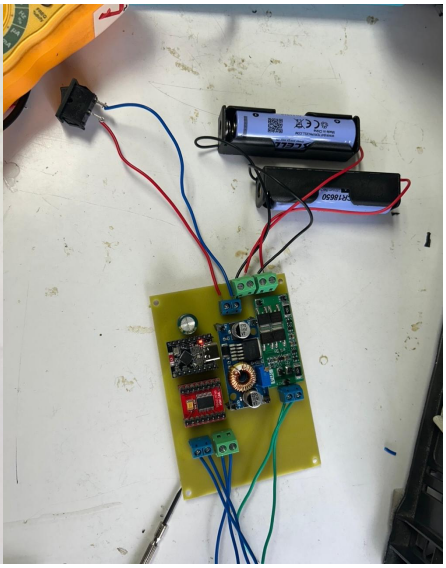
Después de soldar los elementos fijos se instalaron los módulos principales: BMS, XL4005, ESP32-C3 y TB6612FNG. El uso de headers y borneras permitió realizar pruebas por etapas antes de conectar todo el sistema de manera definitiva.



(a) Inicio del ensamblaje.



(b) Prueba del sistema de alimentación.



(c) Módulos instalados.

Figura 15: Ensamblaje progresivo de la PCB.

7.4. Pruebas eléctricas

Antes de realizar las conexiones finales se hicieron pruebas de continuidad y mediciones con multímetro. Se comprobó la alimentación de las baterías y el BMS, se ajustó el XL4005 hasta obtener aproximadamente 5 V y se verificaron las líneas de 5 V y 3.3 V. También se comprobó la alimentación del TB6612FNG y el funcionamiento de los motores.

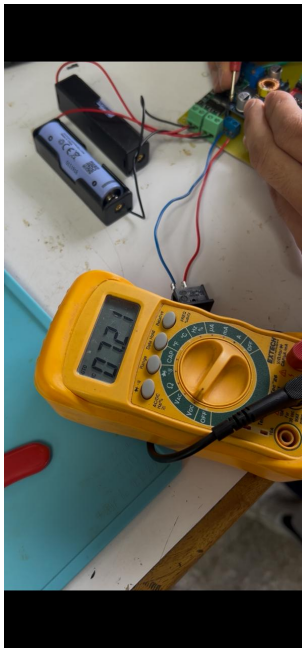


Figura 16: Pruebas eléctricas con multímetro durante el ensamblaje.

7.5. Problemas encontrados y soluciones

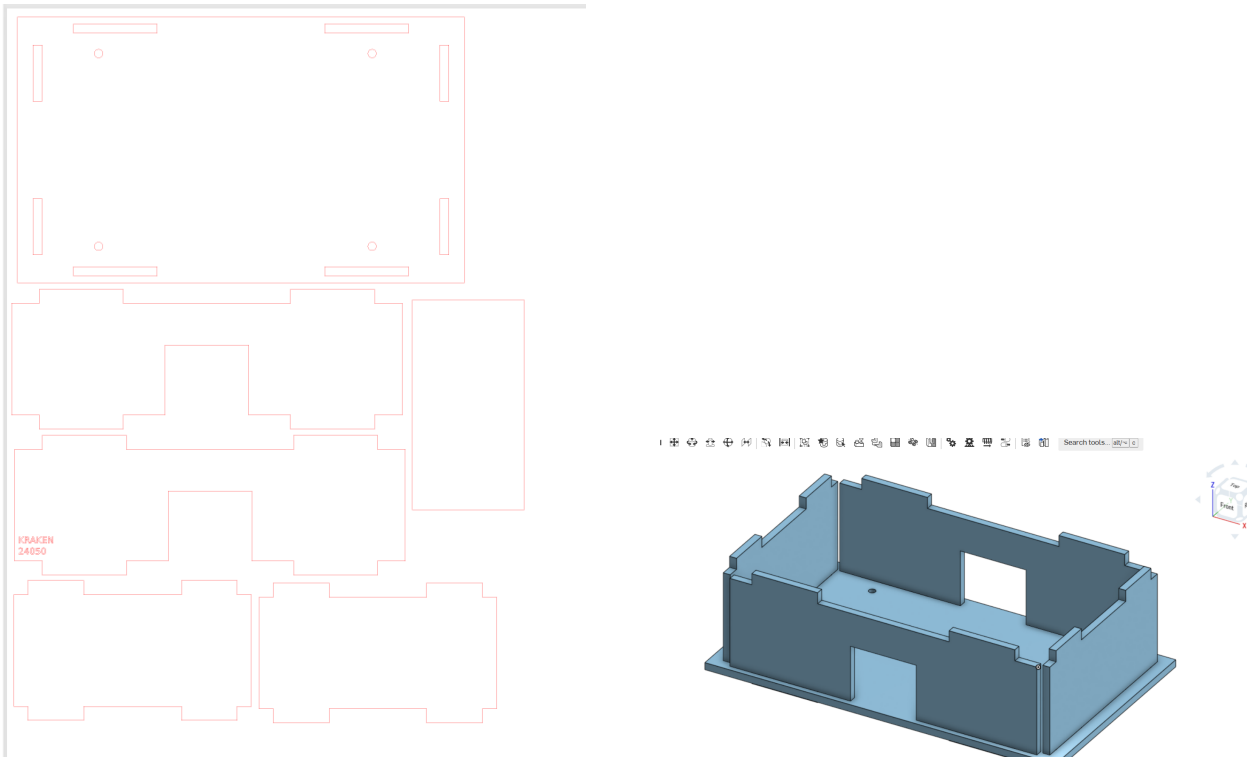
Durante la fabricación y el ensamblaje se presentaron tres problemas principales. Todos fueron corregidos antes de completar el robot.

Cuadro 4: Problemas encontrados durante la etapa final.

Problema	**Solución aplicada**
El primer regulador XL4005 no permitía calibrar correctamente la salida.	Se determinó que el módulo estaba defectuoso y se reemplazó.
Se detectó un corto o intermitencia provocado por residuo de estaño.	Se localizó con ayuda del multímetro y se eliminó el exceso de estaño.
Se levantaron pads durante el trabajo con una bornera.	Se recuperó la conexión mediante un puente de estaño y se verificó continuidad.

7.6. Fabricación e integración del chasis

Las piezas mecánicas se prepararon a partir del diseño 2D y se revisaron en un modelo 3D antes de fabricarlas. Las piezas se cortaron en MDF en el D-Hive. Después se ensambló la estructura y se instalaron la PCB, baterías, motores, ruedas e interruptor.



(a) Diseño 2D de las piezas.

(b) Modelo 3D previo a la fabricación.

Figura 17: Diseño utilizado para fabricar el chasis final.

8. Firmware y software

El control del robot se implementó en un ESP32-C3. Para esta entrega el firmware se documenta como **KRAKEN BLE Control v1.0** y se adjunta en el archivo `KRAKEN_BLE_Control.ino`. El programa está comentado e incluye funciones separadas para cada dirección, control PWM, validación de paquetes BLE y parada automática al perder la conexión.

8.1. Herramientas y dependencias

Cuadro 5: Software y dependencias utilizadas.

Elemento	**Versión / uso**
Arduino IDE	Entorno utilizado para compilar y cargar el firmware
ESP32 Arduino Core	2.0.17
NimBLE-Arduino	2.5.1
Microcontrolador	ESP32-C3 SuperMini
Aplicación móvil	Bluefruit Connect - Control Pad
Comunicación	Bluetooth Low Energy (BLE)

8.2. Funciones principales del firmware

- Avance, retroceso, giro a la izquierda y giro a la derecha.
- Control de dos motores mediante el TB6612FNG.
- Tres niveles de velocidad configurados en el programa: aproximadamente 47 %, 71 % y 100 % del valor PWM disponible.
- Paro manual desde el Control Pad.
- Parada automática de los motores cuando se pierde la conexión BLE.
- Recepción alternativa de comandos de texto F, B, L, R y S mediante el perfil UART BLE.

8.3. Instrucciones de compilación y ejecución

1. Instalar Arduino IDE.
2. Instalar el paquete de placas ESP32 versión 2.0.17.
3. Instalar la biblioteca NimBLE-Arduino versión 2.5.1.
4. Abrir el archivo `KRAKEN_BLE_Control.ino`.
5. Seleccionar la placa ESP32-C3 utilizada en el proyecto y el puerto correspondiente.

6. Compilar y cargar el programa en el ESP32-C3.
7. Abrir Bluefruit Connect en el teléfono y conectarse al dispositivo BLE llamado `KRAKEN`.
8. Utilizar Control Pad para enviar los comandos de movimiento y velocidad.

El código completo se incluye en el Anexo B y también forma parte de los archivos digitales de la entrega.

9. Resultado final y registro de pruebas

Con la PCB instalada dentro del chasis se realizaron pruebas finales de alimentación, comunicación y movimiento. El robot respondió correctamente a los comandos de control y el sistema quedó integrado dentro de la estructura de MDF.

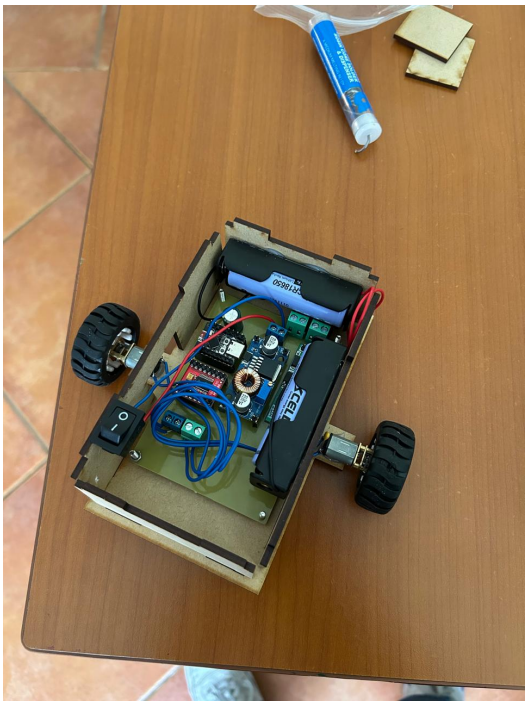


Figura 18: KRAKEN ensamblado con la PCB, baterías, motores, ruedas y switch instalados.

9.1. Registro consolidado de pruebas

Cuadro 6: Registro de pruebas realizadas durante el proyecto.

Prueba	Verificación realizada	Resultado
BLE	Conexión del teléfono con ESP32-C3.	Correcto
Avance y retroceso	Respuesta de ambos motores.	Correcto
Giro izquierda/derecha	Movimiento diferencial de los motores.	Correcto
Velocidad	Cambio de PWM desde el control.	Correcto
Parada por desconexión	Desconexión BLE durante operación.	Correcto
Continuidad PCB	Revisión con multímetro antes de conexiones finales.	Correcto después de correcciones
Regulación	Ajuste de XL4005.	Aproximadamente 5 V
Lógica	Comprobación de alimentación de control.	5 V y 3.3 V verificados
Driver y motores	Alimentación y respuesta del TB6612FNG.	Correcto
Robot ensamblado	Funcionamiento con los componentes dentro del chasis.	Correcto

No se registraron en los reportes mediciones formales de corriente, autonomía total o peso final. Para una siguiente versión sería recomendable incluir estas mediciones como parte de la validación cuantitativa.

9.2. Evidencia en video

- Prototipo funcional: https://youtube.com/shorts/PRTlkrRZpyA?si=jQyeP_-DCtMkYQhl
- Funcionamiento final antes del ensamblaje completo: https://youtube.com/shorts/G6IGOrp_c8g?si=9V9tfObVR4li2Ezq
- Funcionamiento final después del ensamblaje: https://youtube.com/shorts/LHerUUfchp8?si=XLeeXguUP3C7va2U

10. Limitaciones y mejoras futuras

La versión final cumple con el objetivo principal de controlar y mover el robot mediante BLE, pero todavía existen aspectos que pueden mejorarse.

- **Sensores de borde:** los TCRT5000 considerados en la propuesta inicial no se integraron por presupuesto. Una segunda versión podría incorporarlos para reducir el riesgo de salida del Dohyo.

- **Cableado interno:** aunque el sistema funciona, una PCB futura podría reducir todavía más el cableado y ordenar mejor las conexiones hacia baterías y motores.
- **Resistencia mecánica:** el MDF fue adecuado para el prototipo final, pero un material más resistente a impactos podría mejorar la durabilidad en competencia.
- **Pads y borneras:** durante la soldadura se levantaron pads en una bornera. Una nueva versión podría usar pads más robustos, mayor área de cobre o conectores con mejor soporte mecánico.
- **Protección de la PCB:** una fabricación profesional con máscara antisoldante facilitaría el ensamblaje y disminuiría el riesgo de puentes de estaño.
- **Validación cuantitativa:** conviene medir corriente máxima, autonomía de las baterías, temperatura de los módulos y peso final del robot.
- **Integración de prueba:** agregar puntos de prueba claramente identificados permitiría revisar 5 V, 3.3 V, tierra y señales principales con mayor rapidez.

11. Lecciones aprendidas

El proyecto permitió aprender no solo a diseñar una PCB, sino también a comprobar que el diseño pueda fabricarse, soldarse y funcionar dentro de un sistema real.

- Probar primero los módulos en protoboard ayuda a detectar problemas antes de fabricar una PCB.
- El DRC es necesario, pero no reemplaza la inspección visual de los tracks y pads antes de fabricar o soldar.
- El sistema de alimentación debe probarse por separado antes de conectar el microcontrolador y el driver.
- Los capacitores de desacoplamiento y reserva fueron importantes para reducir ruido y caídas de tensión producidas por los motores.
- La limpieza durante la soldadura es importante. Un pequeño residuo de estaño puede provocar un corto difícil de detectar visualmente.
- Aplicar demasiado calor puede levantar pads. Las borneras y conectores requieren cuidado adicional por su masa térmica y esfuerzo mecánico.
- Diseñar con headers y borneras facilita las pruebas por etapas y permite reemplazar módulos defectuosos sin rehacer toda la placa.
- El diseño electrónico y el diseño mecánico deben revisarse en conjunto para asegurar que la PCB, baterías y conexiones realmente quepan dentro del chasis.
- Documentar los cambios durante el proyecto facilita comparar la idea inicial con la versión final y justificar las decisiones tomadas.

12. Recomendaciones para producción a escala

Si KRAKEN se fabricara en varias unidades, se recomienda pasar de una placa fresada de laboratorio a una PCB producida profesionalmente. Esto permitiría incluir máscara antisoldante, serigrafía más clara y agujeros metalizados de forma consistente.

También sería conveniente estandarizar los conectores de motores y baterías, agregar alivio de tensión en los cables, usar fijaciones mecánicas para módulos y baterías, e incluir puntos de prueba para control de calidad. Cada unidad debería pasar por una secuencia de pruebas antes del ensamblaje final: continuidad, ausencia de cortos, verificación de 5 V y 3.3 V, comunicación BLE, prueba individual de cada motor y prueba completa dentro del chasis.

Para una revisión de diseño futura también se recomienda integrar los sensores de borde directamente en la PCB o mediante conectores dedicados, reducir el tamaño de la tarjeta si es posible y mejorar la protección mecánica de los conectores expuestos.

13. Conclusiones

El proyecto KRAKEN se completó desde la propuesta inicial hasta la fabricación y prueba del robot ensamblado. El prototipo en protoboard permitió validar la comunicación BLE, el control de velocidad y los movimientos principales antes de fabricar la PCB. Durante el desarrollo se modificó el sistema de alimentación y se retiraron los sensores de borde de la versión final, manteniendo el objetivo central de un robot controlado desde celular.

La PCB final fue diseñada en Altium Designer, verificada mediante DRC con cero advertencias y cero violaciones y preparada para fabricación mediante archivos Gerber y NC Drill. La placa se fabricó con la ProtoMat S103, se soldó, se probó eléctricamente y se integró dentro del chasis de MDF.

Los problemas encontrados durante el ensamblaje, incluyendo un regulador defectuoso, un corto por residuo de estaño y pads levantados, pudieron corregirse mediante pruebas con multímetro y reparaciones puntuales. Después de estas correcciones el sistema funcionó correctamente con los componentes instalados en el robot.

El resultado final demuestra que el diseño electrónico, el firmware, la PCB y la integración mecánica pueden trabajar como un solo sistema. Las mejoras futuras se concentran principalmente en sensores de borde, robustez mecánica, reducción de cableado y una validación más cuantitativa de corriente, autonomía y peso.

14. Referencias

1. Universidad del Valle de Guatemala. *IE3028 - Proyecto de Curso: Simulación y Fabricación de Circuitos Impresos*. 2026.

2. Espressif Systems. *ESP32-C3-MINI-1 & ESP32-C3-MINI-1U Datasheet*, versión 2.2, 2026. https://documentation.espressif.com/esp32-c3-mini-1_datasheet_en.pdf

3. Espressif Systems. *ESP32-C3 Hardware Design Guidelines*, versión 1.8, 2025. https://docs.espressif.com/projects/esp-hardware-design-guidelines/en/latest/esp32c3/

4. Toshiba Electronic Devices & Storage Corporation. *TB6612FNG: Driver IC for Dual DC Motor*, 2014. https://toshiba.semicon-storage.com/info/TB6612FNG_datasheet_en_20141001.pdf?did=10660&prodName=TB6612FNG

5. Pololu Corporation. *75:1 Micro Metal Gearmotor MP 6V*. https://www.pololu.com/product/2366

6. Vishay Semiconductors. *TCRT5000 and TCRT5000L: Reflective Optical Sensor with Transistor Output*, 2009. https://www.vishay.com/docs/83760/tcrt5000.pdf

7. ANSI IPC-2221A PCB Trace Width Calculator. Utilizada para verificar el ancho de pista del diseño.

8. Universidad del Valle de Guatemala. Especificaciones de fabricación de la LPKF ProtoMat S103 utilizadas durante el diseño del PCB.

9. h2zero. *NimBLE-Arduino*, versión utilizada 2.5.1.

10. Repositorio del proyecto KRAKEN. https://github.com/Joseee-ctrl/KRAKEN-PCB

A. Archivos digitales de la entrega

Además de este PDF, la entrega en Canvas debe incluir el archivo ZIP del proyecto final. El repositorio de respaldo se encuentra en:

https://github.com/Joseee-ctrl/KRAKEN-PCB

Los archivos finales se organizan en las siguientes categorías:

- **Proyecto Altium:** `Esquematicos_Kraken.PrjPcb`.
- **Esquemáticos:** `Kraken.SchDoc`, `Block diagram.SchDoc`, `Power.SchDoc`, `MCU BLE.SchDoc`, `Motor driver.SchDoc`, `Revision history.SchDoc` y carátula.
- **PCB:** `PCBKraken.PcbDoc`.
- **Librería:** `KrakenFootprints.PcbLib` y modelos 3D utilizados por el proyecto.
- **BOM y salidas de Altium:** `Esquematicos_Kraken.BomDoc`, `Esquematicos_Kraken.OutJob`.
- **Verificación:** archivos DRC y reporte HTML con cero violaciones.
- **Fabricación:** Gerbers GTL, GBL y GKO; NC Drill plated y NPTH; archivos CAM/fresado utilizados para la ProtoMat S103.
- **Chasis:** archivos 2D/3D utilizados para fabricar las piezas de MDF.
- **Firmware:** `KRAKEN_BLE_Control.ino`.
- **Documentación y evidencia:** informe final, fotografías y enlaces de video.

B. Firmware completo

El siguiente código corresponde al firmware documentado como KRAKEN BLE Control v1.0. El archivo original se adjunta por separado con la entrega digital.

```
#include <Arduino.h>
#include <NimBLEDevice.h>

/*
  KRAKEN - prototipo electronico
  ESP32-C3 Mini + TB6612FNG + 2 motores N20 de 6 V
  Control desde Bluefruit Connect (Control Pad) por BLE.

  IMPORTANTE:
  - STBY del TB6612 debe conectarse directamente a 3V3.
  - VCC del TB6612 va a 3V3.
  - VM del TB6612 va a la fuente de 6 V.
  - Todos los GND deben estar unidos.
  - La ESP32 se alimenta por USB-C durante esta prueba.
*/

// Conexiones ESP32-C3 -> TB6612FNG
constexpr uint8_t AIN1_PIN = 0; // Motor izquierdo
constexpr uint8_t AIN2_PIN = 1;
constexpr uint8_t PWMA_PIN = 3;
constexpr uint8_t BIN1_PIN = 4; // Motor derecho
constexpr uint8_t BIN2_PIN = 5;
constexpr uint8_t PWMB_PIN = 6;

// Debido a que los motores quedan montados como espejo, normalmente uno debe
// invertirse. Si KRAKEN no avanza recto, cambia true/false aqui o intercambia
// los dos cables del motor correspondiente.
constexpr bool LEFT_REVERSED = false;
constexpr bool RIGHT_REVERSED = true;

uint8_t driveSpeed = 180; // 0-255; arranque moderado (~71 %)

// Nordic UART Service, reconocido por Bluefruit Connect.
constexpr char NUS_SERVICE_UUID[] = "6E400001-B5A3-F393-E0A9-E50E24DCCA9E";
constexpr char NUS_RX_UUID[] = "6E400002-B5A3-F393-E0A9-E50E24DCCA9E";
constexpr char NUS_TX_UUID[] = "6E400003-B5A3-F393-E0A9-E50E24DCCA9E";

void setOneMotor(uint8_t in1, uint8_t in2, uint8_t pwmPin,
                 int16_t command, bool reversed) {
  command = constrain(command, -255, 255);
  if (reversed) command = -command;

  if (command > 0) {
    digitalWrite(in1, HIGH);
    digitalWrite(in2, LOW);
    analogWrite(pwmPin, command);
  } else if (command < 0) {
    digitalWrite(in1, LOW);
    digitalWrite(in2, HIGH);
    analogWrite(pwmPin, -command);
  } else {
    analogWrite(pwmPin, 0);
    digitalWrite(in1, LOW);
    digitalWrite(in2, LOW);
  }
}

void drive(int16_t left, int16_t right) {
  setOneMotor(AIN1_PIN, AIN2_PIN, PWMA_PIN, left, LEFT_REVERSED);
  setOneMotor(BIN1_PIN, BIN2_PIN, PWMB_PIN, right, RIGHT_REVERSED);
}

void stopMotors() {
  // Frenado corto del TB6612: IN1=IN2=HIGH y PWM=HIGH.
  analogWrite(PWMA_PIN, 255);
  analogWrite(PWMB_PIN, 255);
  digitalWrite(AIN1_PIN, HIGH);
  digitalWrite(AIN2_PIN, HIGH);
```

```
  digitalWrite(BIN1_PIN, HIGH);
  digitalWrite(BIN2_PIN, HIGH);
}

void forward() { drive( driveSpeed, driveSpeed); }
void backward() { drive(-driveSpeed, -driveSpeed); }
void turnLeft() { drive(-driveSpeed, driveSpeed); }
void turnRight(){ drive( driveSpeed, -driveSpeed); }

bool validBluefruitPacket(const uint8_t* data, size_t len) {
  if (len != 5 || data[0] != '!' || data[1] != 'B') return false;
  uint8_t checksum = 255;
  for (size_t i = 0; i < len - 1; ++i) checksum -= data[i];
  return checksum == data[len - 1];
}

void processBluefruitButton(uint8_t button, bool pressed) {
  Serial.printf("Boton %u: %s\n", button, pressed ? "presionado" : "soltado");

  // Botones numericos: 1, 2 y 3 eligen velocidad; 4 es paro.
  if (pressed) {
    if (button == 1) { driveSpeed = 120; Serial.println("Velocidad: 47 %"); return; }
    if (button == 2) { driveSpeed = 180; Serial.println("Velocidad: 71 %"); return; }
    if (button == 3) { driveSpeed = 255; Serial.println("Velocidad: 100 %"); return; }
    if (button == 4) { stopMotors(); Serial.println("PARO"); return; }
  }

  // Bluefruit Control Pad: 5=izquierda, 6=derecha, 7=abajo, 8=arriba.
  if (!pressed) {
    if (button >= 5 && button <= 8) stopMotors();
    return;
  }

  switch (button) {
    case 5: turnLeft(); break;
    case 6: turnRight(); break;
    case 7: backward(); break;
    case 8: forward(); break;
    default: break;
  }
}

// Permite probar tambien desde la pestaña UART escribiendo F, B, L, R o S.
void processTextCommand(const uint8_t* data, size_t len) {
  if (len == 0) return;
  char command = toupper(static_cast<char>(data[0]));
  switch (command) {
    case 'F': forward(); break;
    case 'B': backward(); break;
    case 'L': turnLeft(); break;
    case 'R': turnRight(); break;
    case 'S': stopMotors(); break;
    default: return;
  }
  Serial.printf("Comando UART: %c\n", command);
}

class RxCallbacks : public NimBLECharacteristicCallbacks {
  void onWrite(NimBLECharacteristic* characteristic,
               NimBLEConnInfo& connInfo) override {
    NimBLEAttValue value = characteristic->getValue();
    const uint8_t* data = value.data();
    const size_t len = value.size();

    if (validBluefruitPacket(data, len)) {
      const uint8_t button = data[2] - '0';
      const bool pressed = data[3] == '1';
      processBluefruitButton(button, pressed);
    } else {
      processTextCommand(data, len);
    }
  }
};

class ServerCallbacks : public NimBLEServerCallbacks {
```

```
  void onConnect(NimBLEServer* server, NimBLEConnInfo& connInfo) override {
    Serial.println("Telefono conectado por BLE");
  }

  void onDisconnect(NimBLEServer* server, NimBLEConnInfo& connInfo,
                    int reason) override {
    stopMotors();
    Serial.println("Telefono desconectado: motores detenidos");
    NimBLEDevice::startAdvertising();
  }
};

RxCallbacks rxCallbacks;
ServerCallbacks serverCallbacks;

void setup() {
  Serial.begin(115200);

  pinMode(AIN1_PIN, OUTPUT);
  pinMode(AIN2_PIN, OUTPUT);
  pinMode(PWMA_PIN, OUTPUT);
  pinMode(BIN1_PIN, OUTPUT);
  pinMode(BIN2_PIN, OUTPUT);
  pinMode(PWMB_PIN, OUTPUT);

  analogWriteResolution(PWMA_PIN, 8);
  analogWriteResolution(PWMB_PIN, 8);
  analogWriteFrequency(PWMA_PIN, 20000);
  analogWriteFrequency(PWMB_PIN, 20000);
  stopMotors();

  NimBLEDevice::init("KRAKEN");
  NimBLEServer* server = NimBLEDevice::createServer();
  server->setCallbacks(&serverCallbacks, false);

  NimBLEService* uartService = server->createService(NUS_SERVICE_UUID);

  // TX debe existir para que Bluefruit reconozca el perfil UART, aunque este
  // prototipo solo necesita recibir comandos en RX.
  uartService->createCharacteristic(
      NUS_TX_UUID, NIMBLE_PROPERTY::READ | NIMBLE_PROPERTY::NOTIFY);

  NimBLECharacteristic* rxCharacteristic = uartService->createCharacteristic(
      NUS_RX_UUID, NIMBLE_PROPERTY::WRITE | NIMBLE_PROPERTY::WRITE_NR);
  rxCharacteristic->setCallbacks(&rxCallbacks);

  uartService->start();

  NimBLEAdvertising* advertising = NimBLEDevice::getAdvertising();
  advertising->addServiceUUID(NUS_SERVICE_UUID);
  advertising->enableScanResponse(true);
  NimBLEDevice::startAdvertising();

  Serial.println("KRAKEN listo. Busca KRAKEN en Bluefruit Connect.");
}

void loop() {
  delay(20);
}
```

C. Evidencia multimedia

Evidencia	Enlace
Prototipo en protoboard	https://youtube.com/shorts/PRTlkrRZpyA?si=jQyeP_-DCtMkYQhl
Funcionamiento final antes del ensamblaje	https://youtube.com/shorts/G6IGOrp_c8g?si=9V9tfObVR4li2Ezq
Funcionamiento final ensamblado	https://youtube.com/shorts/LHerUUfchp8?si=XLeeXguUP3C7va2U
Repositorio GitHub	https://github.com/Joseee-ctrl/KRAKEN-PCB